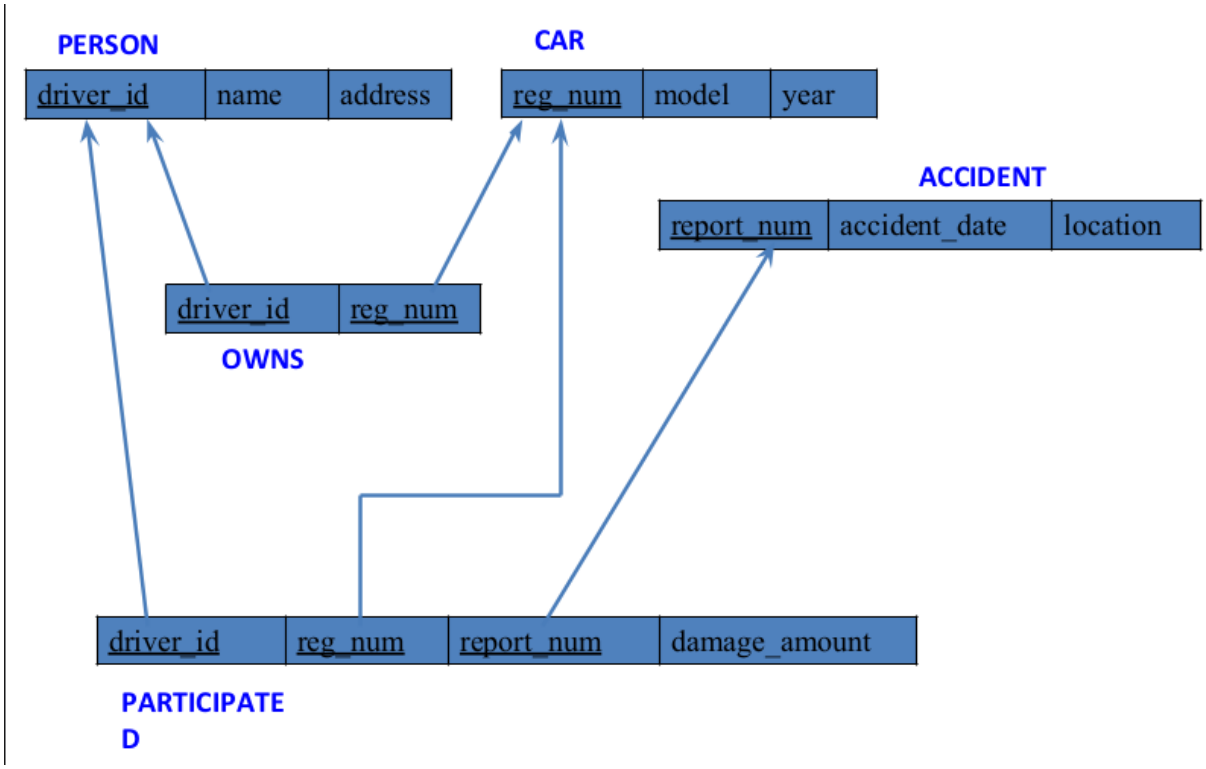
WEEK 1 - INSURANCE DATABASE

Question:

Consider the Insurance database given below.
PERSON (driver_id: String, name: String, address: String)
CAR (reg_num: String, model: String, year: int)
ACCIDENT (report_num: int, accident_date: date, location: String)
OWNS (driver_id: String, reg_num: String)
PARTICIPATED (driver_id: String,reg_num: String, report_num: int, damage_amount: int)
i. Create the above tables by properly specifying the primary keys and the foreign keys.
ii. Enter at least five tuples for each relation
iii. Display Accident date and location
iv. Update the damage amount to 25000 for the car with a specific reg_num (example 'KA053408') for which the accident report number was 12.
v. Add a new accident to the database.
vi. Display Accident date and location
vii. Display driver id who did accident with damage amount greater than or equal to Rs.25000

Schema Diagram:



Create database

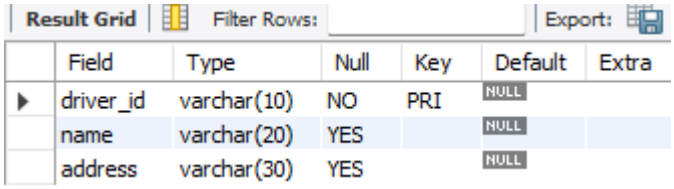
```
create database insurance;
use insurance;
```

Create table

```
create table person(
driver_id varchar(20),
name varchar(30),
address varchar(50),
PRIMARY KEY(driver_id));

create table car(
reg_num varchar(15),
model varchar(10),
year int,
PRIMARY KEY(reg_num));

create table owns(driver_id varchar(20),
reg_num varchar(10),
PRIMARY KEY(driver_id, reg_num),
FOREIGN KEY(driver_id) REFERENCES person(driver_id),
FOREIGN KEY(reg_num) REFERENCES car(reg_num));


create table accident(report_num int,
accident_date date,
location varchar(50),
PRIMARY KEY(report_num));


create table participated(
driver_id varchar(20),
reg_num varchar(10),
report_num int,
damage_amount int,
PRIMARY KEY(driver_id,reg_num,report_num),
FOREIGN KEY(driver_id) REFERENCES person(driver_id),
FOREIGN KEY(reg_num) REFERENCES car(reg_num),
FOREIGN KEY(report_num) REFERENCES accident(report_num));
```

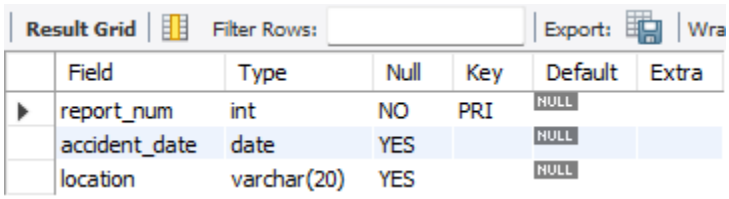
Structure Of Tables:

desc person;



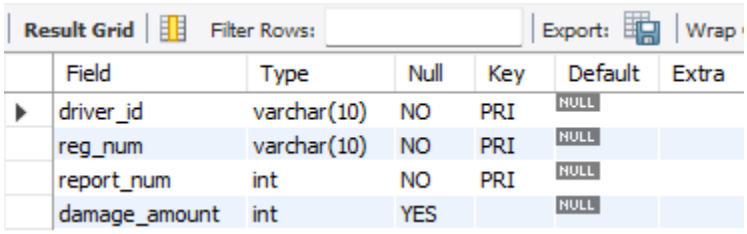
Field	Type	Null	Key	Default	Extra
driver_id	varchar(10)	NO	PRI	NULL	
name	varchar(20)	YES		NULL	
address	varchar(30)	YES		NULL	

desc accident;



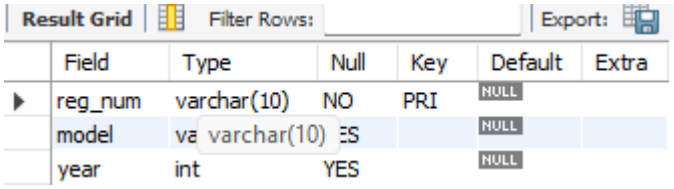
Field	Type	Null	Key	Default	Extra
report_num	int	NO	PRI	NULL	
accident_date	date	YES		NULL	
location	varchar(20)	YES		NULL	

desc participated;



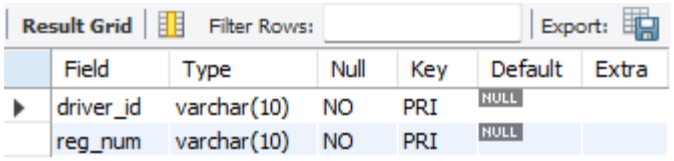
Field	Type	Null	Key	Default	Extra
driver_id	varchar(10)	NO	PRI	NULL	
reg_num	varchar(10)	NO	PRI	NULL	
report_num	int	NO	PRI	NULL	
damage_amount	int	YES		NULL	

desc car;



Field	Type	Null	Key	Default	Extra
reg_num	varchar(10)	NO	PRI	NULL	
model	varchar(10)	YES		NULL	
year	int	YES		NULL	

desc owns;

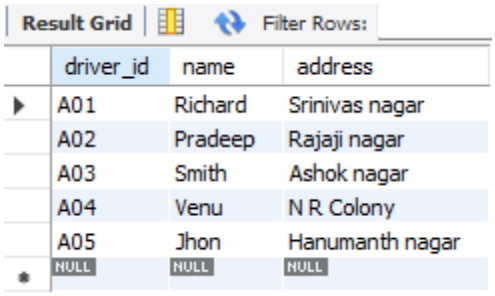


Field	Type	Null	Key	Default	Extra
driver_id	varchar(10)	NO	PRI	NULL	
reg_num	varchar(10)	NO	PRI	NULL	

Insertion of Values:

insert into person values(A01,'Richard','Srinivas nagar');
insert into person values(A02, 'Pradeep','Rajaji nagar');
insert into person values(A03, 'Smith','Ashok nagar');
insert into person values(A04, 'Venu','N R Colony');
insert into person values(A05, 'John','Hanumanth nagar');

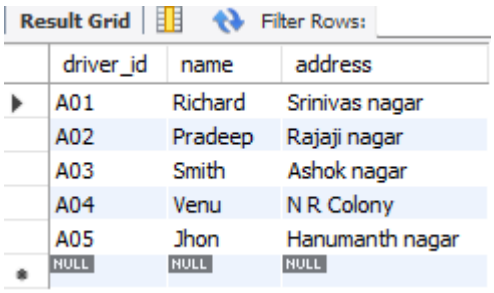
select * from person;



driver_id	name	address
A01	Richard	Srinivas nagar
A02	Pradeep	Rajaji nagar
A03	Smith	Ashok nagar
A04	Venu	N R Colony
A05	Jhon	Hanumanth nagar
NULL	NULL	NULL

insert into car values(KA052250,Indica,1990);
insert into car values(KA031181, Lancer,1957);
insert into car values(KA095477,Toyota,1998);
insert into car values(KA053408,Honda, 2008);
insert into car values(KA041702, Audi, 2005);

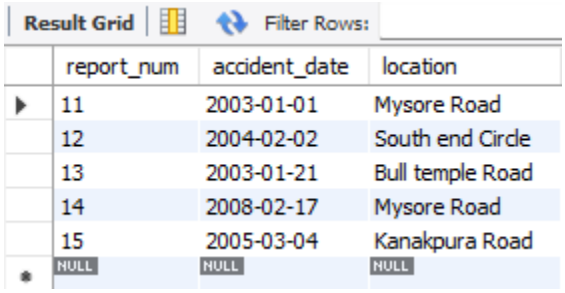
select * from car;



driver_id	name	address
A01	Richard	Srinivas nagar
A02	Pradeep	Rajaji nagar
A03	Smith	Ashok nagar
A04	Venu	N R Colony
A05	Jhon	Hanumanth nagar
NULL	NULL	NULL

insert into accident values('11', '2003-01-01', 'Mysore Road');
insert into accident values('12', '2004-02-02', 'South End Circle');
insert into accident values('13', '2003-01-21', 'Bull Temple Road');
insert into accident values('14', '2008-02-17', 'Mysore Road');
insert into accident values('15', '2005-03-04', 'Kanakpura Road');

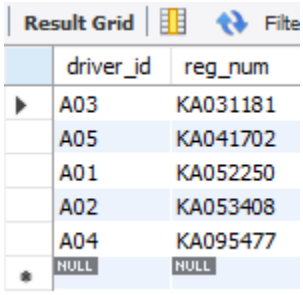
select * from accident;



report_num	accident_date	location
11	2003-01-01	Mysore Road
12	2004-02-02	South end Circle
13	2003-01-21	Bull temple Road
14	2008-02-17	Mysore Road
15	2005-03-04	Kanakpura Road
NULL	NULL	NULL

insert into owns values('A01', 'KA052250');
insert into owns values('A02', 'KA053408');
insert into owns values('A02', 'KA031181');
insert into owns values('A04', 'KA095477');
insert into owns values('A05', 'KA041702');

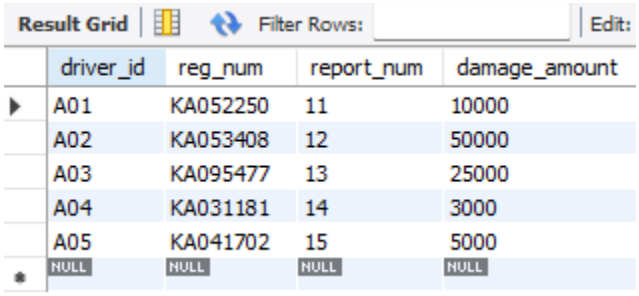
select * from owns;



driver_id	reg_num
A03	KA031181
A05	KA041702
A01	KA052250
A02	KA053408
A04	KA095477
NULL	NULL

insert into participated values('A01', 'KA052250', '11', '10000');
insert into participated values('A02', 'KA053408', '12', '50000');
insert into participated values('A03', 'KA095477', '13', '25000');
insert into participated values('A04', 'KA031181', '14', '3000');
insert into participated values('A05', 'KA041702', '15', '5000');

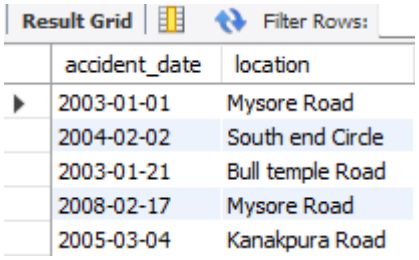
select * from participated;



driver_id	reg_num	report_num	damage_amount
A01	KA052250	11	10000
A02	KA053408	12	50000
A03	KA095477	13	25000
A04	KA031181	14	3000
A05	KA041702	15	5000
NULL	NULL	NULL	NULL

QUERY: Display Accident date and location

select accident_date,location from accident;

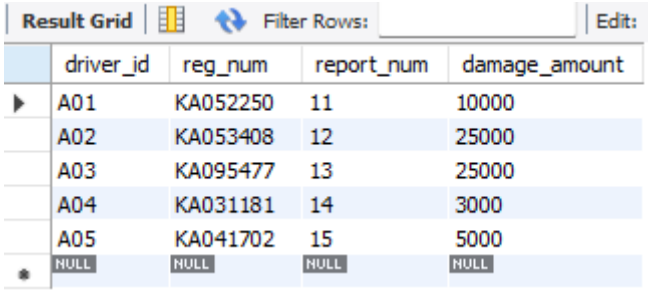


accident_date	location
2003-01-01	Mysore Road
2004-02-02	South end Circle
2003-01-21	Bull temple Road
2008-02-17	Mysore Road
2005-03-04	Kanakpura Road

QUERY: Update the damage amount to 25000 for the car with a specific reg_num (example 'KA053408') for which the accident report number was 12.

update participated set damage_amount=25000 where reg_num='KA053408' and report_num=12;

select * from participated;



driver_id	reg_num	report_num	damage_amount
A01	KA052250	11	10000
A02	KA053408	12	25000
A03	KA095477	13	25000
A04	KA031181	14	3000
A05	KA041702	15	5000
NULL	NULL	NULL	NULL

QUERY: Add a new accident to the database.

```
insert into accident values(16,15/03/2008,'domblur');
```

```
select * from accident;
```

Result Grid | Filter Rows:

report_num	accident_date	location
11	2003-01-01	Mysore Road
12	2004-02-02	South end Circle
13	2003-01-21	Bull temple Road
14	2008-02-17	Mysore Road
15	2005-03-04	Kanakpura Road
16	0000-00-00	domblur
NULL	NULL	NULL

QUERY: Display driver id who did accident with damage amount greater than or equal to Rs.25000

```
select driver_id from participated where damage_amount>=25000;
```

driver_id
A02
A03

WEEK 2 – Additional Queries Insurance Database

1)Display the entire CAR relation in the ascending order of manufacturing year.

select * from car order by year asc;

reg_num	model	year
KA031181	Lancer	1957
KA052250	Indica	1990
KA095477	Toyota	1998
KA041702	Audi	2005
KA053408	Honda	2008
NULL	NULL	NULL

2)Find the number of accidents in which cars belonging to a specific model (example 'Lancer') were involved.

select count (report_num) from car c,participated p
where c.reg_num=p.reg_num and c.model=”lancer”;

Result Grid

count(report_num)
1

3)Find the total number of people who owned cars that were involved in accidents in 2008.

select count(distinct driver_id) from participated p,accident a where p.report_num=a.report_num and a.accident_date like"__08%";

count(distinct driver_id)
1

4) List the entire participated relation in the descending order of damage amount.
select *from participated order by damage_amount desc;

driver_id	reg_num	Report_num	damage_amount
A02	KA053408	12	50000
A03	KA095477	13	25000
A01	KA052250	11	10000
A05	KA041702	15	5000
A04	KA031181	14	3000
NULL	NULL	NULL	NULL

5) Find The Average Damage Amount
select avg(damage_amount) from participated;

avg(damage_amount)
18600.0000

6)DELETE THE TUPLE WHOSE DAMAGE AMOUNT IS BELOW THE AVERAGE DAMAGE AMOUNT

delete from participated where damage_amount<18600;
select *from participated;

driver_id	reg_num	Report_num	damage_amount
A02	KA053408	12	50000
A03	KA095477	13	25000
NULL	NULL	NULL	NULL

7)List the name of drivers whose damage is greater than the average damage amount.

select name from person a, participated b WHERE a.driver_id = b.driver_id and damage_amount > (select avg(damage_amount) from participated);

name
Pradeep

8)Find Maximum Damage Amount.

select max(damage_amount) from participated;

max(damage_amount)
50000

WEEK 3 - BANK DATABASE

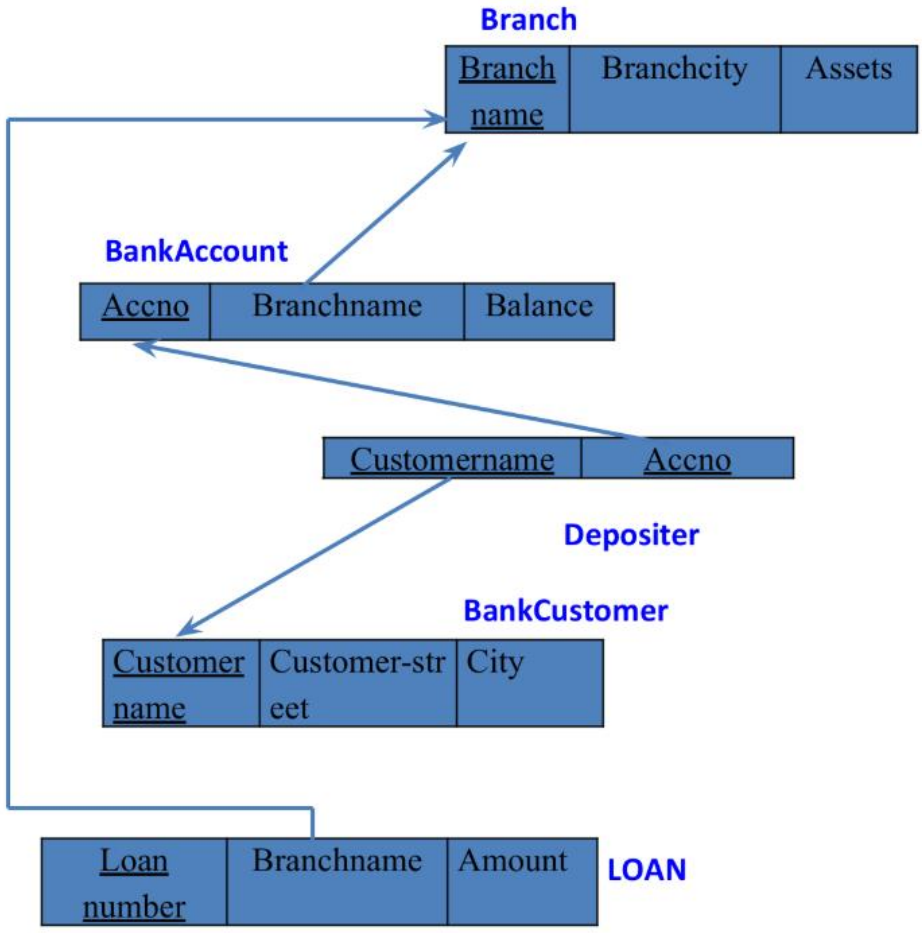
QUESTION:

Consider the following database for a banking enterprise.

Branch (branch-name: String, branch-city: String, assets: real)
BankAccount(accno: int, branch-name: String, balance: real)
BankCustomer (customer-name: String, customer-street: String, customer-city: String)
Depositer(customer-name: String, accno: int)
Loan (loan-number: int, branch-name: String, amount: real)

i. Create the above tables by properly specifying the primary keys and the foreign keys.
ii. Enter at least five tuples for each relation.
iii. Find all the customers who have at least two accounts at the Main branch (ex. SBI_ResidencyRoad).
iv. Find all the customers who have an account at all the branches located in a specific city (Ex. Delhi).
v. Demonstrate how you delete all account tuples at every branch located in a specific city (Ex. Bombay).

SCHEMA DIAGRAM:



CREATION OF DATABASE

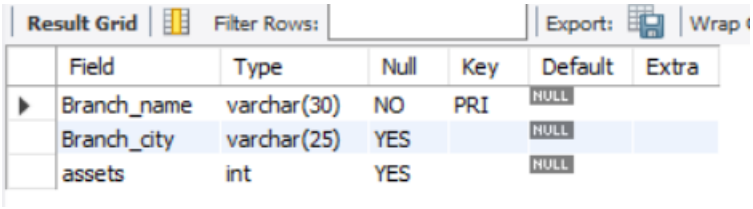
```
create database bms_bank;
use bms_bank;
```

CREATION OF TABLES

```
create table branch(
   Branch_name varchar(30),
   Branch_city varchar(25),
   assets int,
   PRIMARY KEY (Branch_name));


create table BankAccount(
   Accno int,
   Branch_name varchar(30),
   Balance int,
   PRIMARY KEY(Accno),
   foreign key (Branch_name) references branch(Branch_name));


create table BankCustomer(
   Customername varchar(20),
   Customer_street varchar(30),
   CustomerCity varchar (35),
   PRIMARY KEY(Customername));


create table Depositer(
   Customername varchar(20),
   Accno int,
   PRIMARY KEY(Customername,Accno),
   foreign key (Accno) references BankAccount(Accno),
   foreign key (Customername) references BankCustomer(Customername));


create table Loan(
   Loan_number int,
   Branch_name varchar(30),
   Amount int,
   PRIMARY KEY(Loan_number),
   foreign key (Branch_name) references branch(Branch_name));
```

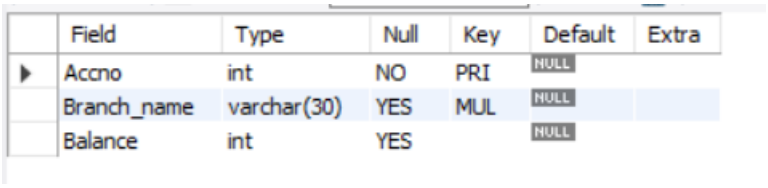
STRUCTURE OF TABLES:

desc Branch;



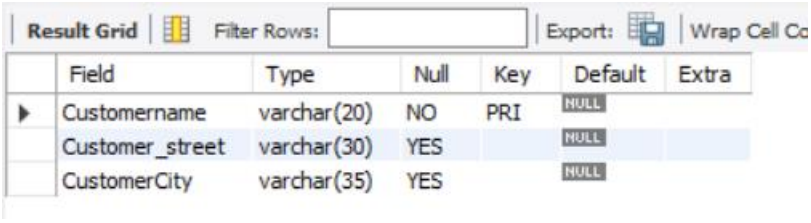
Field	Type	Null	Key	Default	Extra
Branch_name	varchar(30)	NO	PRI	NULL	
Branch_city	varchar(25)	YES		NULL	
assets	int	YES		NULL	

desc BankAccount;



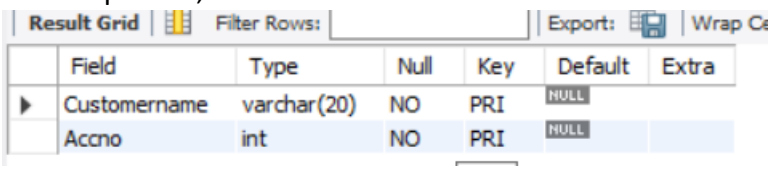
Field	Type	Null	Key	Default	Extra
Accno	int	NO	PRI	NULL	
Branch_name	varchar(30)	YES	MUL	NULL	
Balance	int	YES		NULL	

desc BankCustomer;



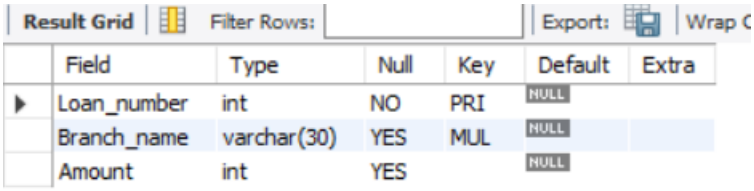
Field	Type	Null	Key	Default	Extra
Customername	varchar(20)	NO	PRI	NULL	
Customer_street	varchar(30)	YES		NULL	
CustomerCity	varchar(35)	YES		NULL	

desc Depositor;



Field	Type	Null	Key	Default	Extra
Customername	varchar(20)	NO	PRI	NULL	
Accno	int	NO	PRI	NULL	

desc Loan;

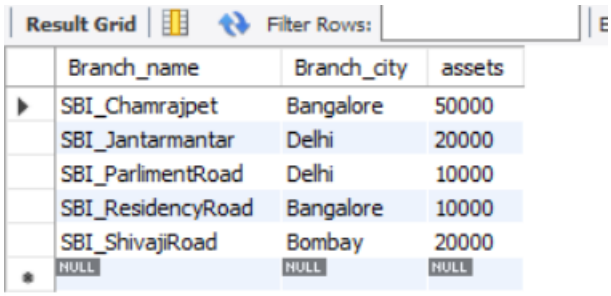


Field	Type	Null	Key	Default	Extra
Loan_number	int	NO	PRI	NULL	
Branch_name	varchar(30)	YES	MUL	NULL	
Amount	int	YES		NULL	

INSERTION OF VALUES:

```
insert into branch values('SBI_Chamrajpet','Bangalore',50000);
insert into branch values('SBI_ResidencyRoad','Bangalore',10000);
insert into branch values('SBI_ShivajiRoad','Bombay',20000);
insert into branch values('SBI_ParlimentRoad','Delhi',10000);
insert into branch values('SBI_Jantarmantar','Delhi',20000);
```

select * from branch;

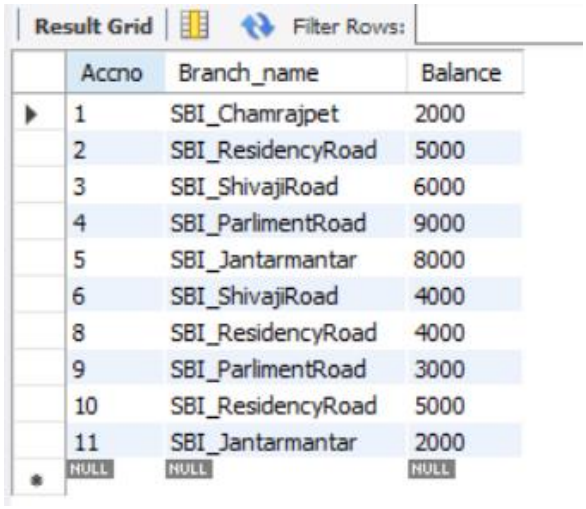


Branch_name	Branch_city	assets
SBI_Chamrajpet	Bangalore	50000
SBI_Jantarmantar	Delhi	20000
SBI_ParlimentRoad	Delhi	10000
SBI_ResidencyRoad	Bangalore	10000
SBI_ShivajiRoad	Bombay	20000
NULL	NULL	NULL

---

insert into BankAccount values(1,'SBI_Chamrajpet',2000);
insert into BankAccount values(2,'SBI_ResidencyRoad',5000);
insert into BankAccount values(3,'SBI_ShivajiRoad',6000);
insert into BankAccount values(4,'SBI_ParlimentRoad',9000);
insert into BankAccount values(5,'SBI_Jantarmantar',8000);
insert into BankAccount values(6,'SBI_ShivajiRoad',4000);
insert into BankAccount values(8,'SBI_ResidencyRoad',4000);
insert into BankAccount values(9,'SBI_ParlimentRoad',3000);
insert into BankAccount values(10,'SBI_ResidencyRoad',5000);
insert into BankAccount values(11,'SBI_Jantarmantar',2000);

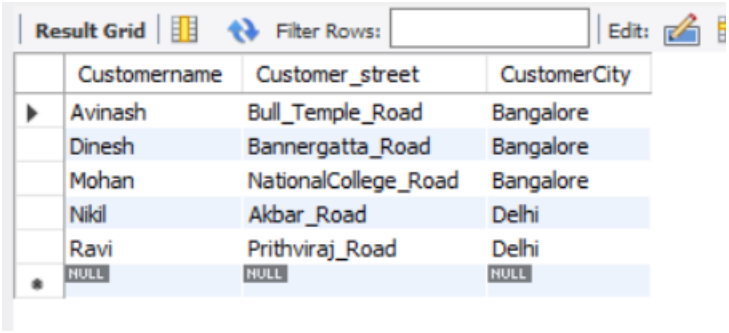
select * from BankAccount;



Accno	Branch_name	Balance
1	SBI_Chamrajpet	2000
2	SBI_ResidencyRoad	5000
3	SBI_ShivajiRoad	6000
4	SBI_ParlimentRoad	9000
5	SBI_Jantarmantar	8000
6	SBI_ShivajiRoad	4000
8	SBI_ResidencyRoad	4000
9	SBI_ParlimentRoad	3000
10	SBI_ResidencyRoad	5000
11	SBI_Jantarmantar	2000
NULL	NULL	NULL

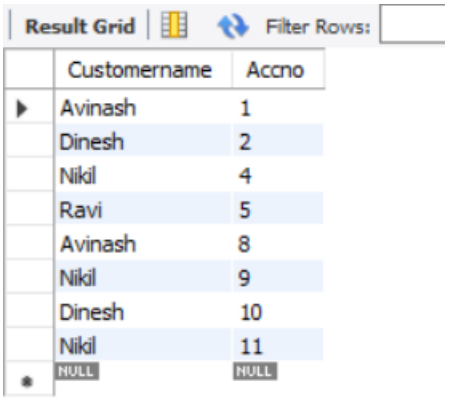
insert into BankCustomer values('Avinash','Bull_Temple_Road','Bangalore');
insert into BankCustomer values('Dinesh','Bannergatta_Road','Bangalore');
insert into BankCustomer values('Mohan','NationalCollege_Road','Bangalore');
insert into BankCustomer values('Nikil','Akbar_Road','Delhi');
insert into BankCustomer values('Ravi','Prithviraj_Road','Delhi');

select * from BankCustomer;



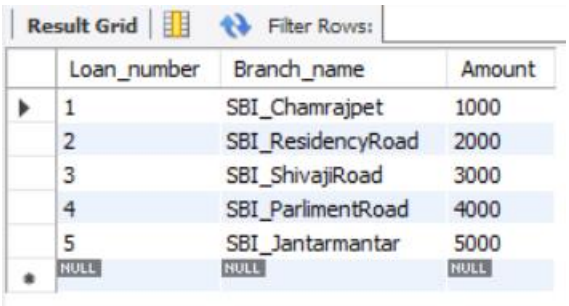
Customername	Customer_street	CustomerCity
Avinash	Bull_Temple_Road	Bangalore
Dinesh	Bannergatta_Road	Bangalore
Mohan	NationalCollege_Road	Bangalore
Nikil	Akbar_Road	Delhi
Ravi	Prithviraj_Road	Delhi
NULL	NULL	NULL

```
insert into Depositer values('Avinash',1);
insert into Depositer values('Dinesh',2);
insert into Depositer values('Nikil',4);
insert into Depositer values('Ravi',5);
insert into Depositer values('Avinash',8);
insert into Depositer values('Nikil',9);
insert into Depositer values('Dinesh',10);
insert into Depositer values('Nikil',11);

select * from Depositer;
```



Customername	Accno
Avinash	1
Dinesh	2
Nikil	4
Ravi	5
Avinash	8
Nikil	9
Dinesh	10
Nikil	11
NULL	NULL

```
insert into Loan values(1,'SBI_Chamrajpet',1000);
insert into Loan values(2,'SBI_ResidencyRoad',2000);
insert into Loan values(3,'SBI_ShivajiRoad',3000);
insert into Loan values(4,'SBI_ParlimentRoad',4000);
insert into Loan values(5,'SBI_Jantarmantar',5000);
select * from Loan;
```



Loan_number	Branch_name	Amount
1	SBI_Chamrajpet	1000
2	SBI_ResidencyRoad	2000
3	SBI_ShivajiRoad	3000
4	SBI_ParlimentRoad	4000
5	SBI_Jantarmantar	5000
NULL	NULL	NULL

QUERY 3: Display the branch name and assets from all branches in lakhs of rupees and rename the assets column to 'assets in lakhs'.

select Branch_name, CONCAT(assets/100000,'lakhs')assets_in_lakhs from branch;

Result Grid | Filter Rows:

Branch_name	assets_in_lakhs
SBI_Chamrajpet	0.5000lakhs
SBI_Jantarmantar	0.2000lakhs
SBI_ParlimentRoad	0.1000lakhs
SBI_ResidencyRoad	0.1000lakhs
SBI_ShivajiRoad	0.2000lakhs

QUERY 4: Find all the customers who have at least two accounts at the Same branch (ex. SBI_ResidencyRoad).

select d.Customername from Depositer d, BankAccount b where
b.Branch_name='SBI_ResidencyRoad' and d.Accno=b.Accno group by
d.Customername having count(d.Accno)>=2;

Customername
Dinesh

QUERY 5: Create a view which gives each branch the sum of the amount of all the loans at the branch.

create view sum_of_loan as select Branch_name, SUM(Balance) from BankAccount
group by Branch_name;

select * from sum_of_loan;

Result Grid | Filter Rows:

Branch_name	SUM(Balance)
SBI_Chamrajpet	2000
SBI_Jantarmantar	10000
SBI_ParlimentRoad	12000
SBI_ResidencyRoad	14000
SBI_ShivajiRoad	10000

WEEK 4 – Additional Queries (Bank Database)

QUERY 6: Update the annual interest payments are made and all branches are to be increased by 5%.

>update bankaccount set balance=balance*1.05;

Branch_name	Branch_city	assets
SBI_Chamrajpet	Bangalore	50000
SBI_Jantarmantar	Delhi	20000
SBI_ParlimentRoad	Delhi	10000
SBI_ResidencyRoad	Bangalore	10000
SBI_ShivajiRoad	Bombay	20000
NULL	NULL	NULL

QUERY 7: Find all the customer who have account at all the branches located in specific city.

```
select distinct S.customername from depositer as S
where not exists ( (select branch_name from branch where branch_city = 'Delhi')
                   except (select R.branch_name from depositer as T, bankaccount as
                    where T.Accno = R.Accno and S.customername = T.customername));
```

customername
Nikil

QUERY 8: Demonstrate how you delete all account tuples at every branch located in a specific city (Ex.Bombay)

```
delete from bankaccount where branch_name in ( select branch_name from branch
where branch_city = 'Bombay');
```

select *from bankaccount;

Accno	Branch_name	Balance
2	SBI_ResidencyRoad	5250
3	SBI_ShivajiRoad	6300
4	SBI_ParlimentRoad	9450
5	SBI_Jantarmantar	8400
6	SBI_ShivajiRoad	4200
8	SBI_ResidencyRoad	4200
9	SBI_ParlimentRoad	3150
10	SBI_ResidencyRoad	5250
11	SBI_Jantarmantar	2100
NULL	NULL	NULL

QUERY 9: List the entire loan relation in the descending order of amount.

> SELECT * FROM LOAN ORDER BY AMOUNT DESC;

Loan_number	Branch_name	Amount
5	SBI_Jantarmantar	5000
4	SBI_ParlimentRoad	4000
3	SBI_ShivajiRoad	3000
2	SBI_ResidencyRoad	2000
1	SBI_Chamrajpet	1000
NULL	NULL	NULL

QUERY 10: Find all customers who have both an account and loan at Bangalore branch.

```
select distinct b.borrower_name from borrower b
join depositor d on b.borrower_name = d.customername
join bankaccount ba on d.accno = ba.accno
join branch br on ba.branch_name = br.branch_name
where br.branch_city = 'Bangalore';
```

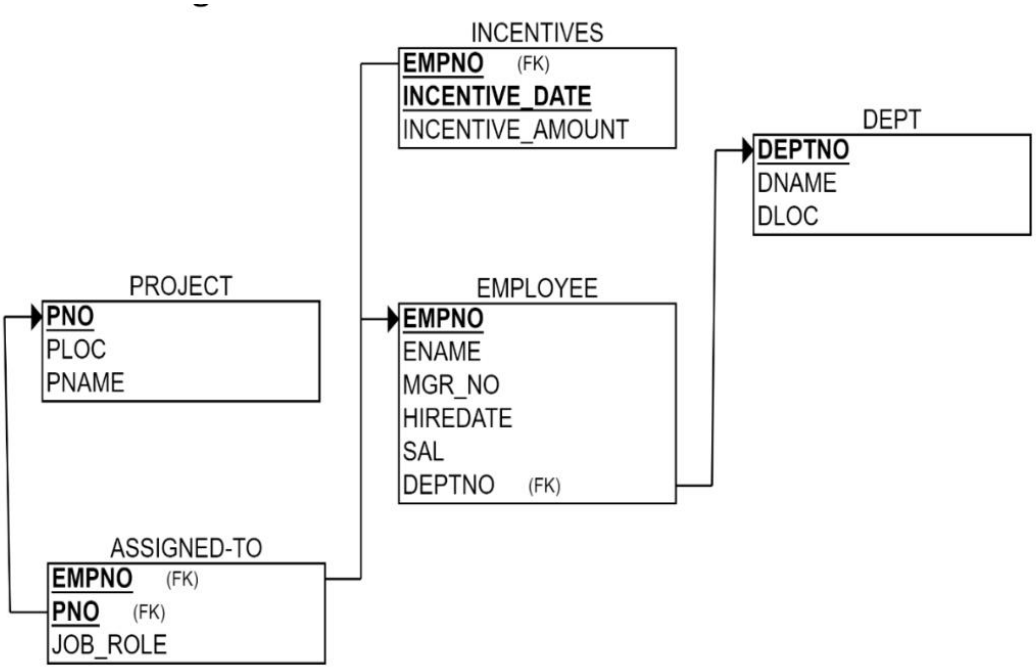
borrower_name
Avinash
Dinesh

WEEK 5 – EMPLOYEE DATABASE

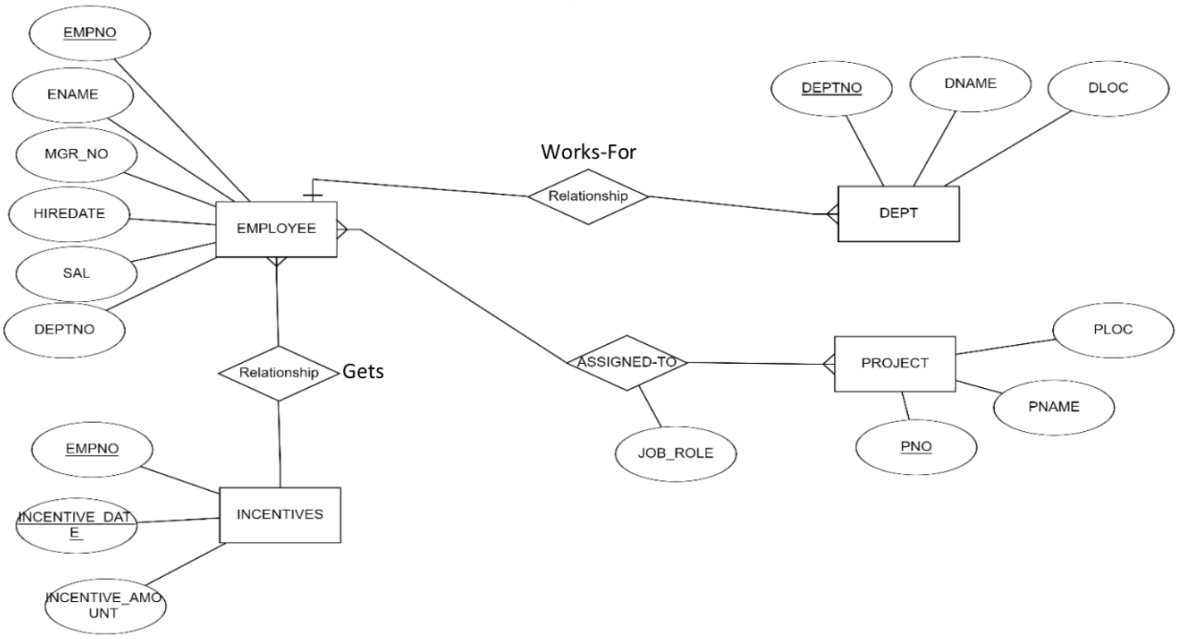
QUESTION:

1. Using Scheme diagram, Create tables by properly specifying the primary keys and the foreign keys.
2. Enter greater than five tuples for each table.
3. Retrieve the employee numbers of all employees who work on project located in Bengaluru, Hyderabad, or Mysuru
4. Get Employee ID's of those employees who didn't receive incentives
5. Write a SQL query to find the employees name, number, dept, job_role, department location and project location who are working for a project location same as his/her department location.

SCHEMA DIAGRAM



ER – DIAGRAM



CREATION OF DATABASE:

```
create database emp;
use emp;
```

CREATION OF TABLES:

```
CREATE TABLE dept (
  deptno decimal(2,0) primary key,
  dname varchar(14) default NULL,
  loc varchar(13) default NULL
);

CREATE TABLE emp (
  empno decimal(4,0) primary key,
  ename varchar(10) default NULL,
  mgr_no decimal(4,0) default NULL,
  hiredate date default NULL,
  sal decimal(7,2) default NULL,
  deptno decimal(2,0) references dept(deptno) on delete cascade on update cascade
);

create table incentives (
  empno decimal(4,0) references emp(empno) on delete cascade on update cascade,
  incentive_date date,
  incentive_amount decimal(10,2),
  primary key(empno,incentive_date)
  );
```

```
Create table project (
 pno int primary key,
 pname varchar(30) not null,
 ploc varchar(30)
 );

Create table assigned_to (
 empno decimal(4,0) references emp(empno) on delete cascade on update cascade,
 pno int references project(pno) on delete cascade on update cascade,
 job_role varchar(30),
 primary key(empno,pno)
 );
```

STRUCTURE OF TABLES:

desc dept;

Field	Type	Null	Key	Default	Extra
deptno	decimal(2,0)	NO	PRI	NULL	
dname	varchar(14)	YES		NULL	
loc	varchar(13)	YES		NULL	

desc emp;

Field	Type	Null	Key	Default	Extra
empno	decimal(4,0)	NO	PRI	NULL	
ename	varchar(10)	YES		NULL	
mgr_no	decimal(4,0)	YES		NULL	
hiredate	date	YES		NULL	
sal	decimal(7,2)	YES		NULL	
deptno	decimal(2,0)	YES		NULL	

desc incentives;

Field	Type	Null	Key	Default	Extra
empno	decimal(4,0)	NO	PRI	NULL	
incentive_date	date	NO	PRI	NULL	
incentive_amount	decimal(10,2)	YES		NULL	

desc project;

Field	Type	Null	Key	Default	Extra
pno	int	NO	PRI	NULL	
pname	varchar(30)	NO		NULL	
ploc	varchar(30)	YES		NULL	

desc assigned_to;

Field	Type	Null	Key	Default	Extra
empno	decimal(4,0)	NO	PRI	NULL	
pno	int	NO	PRI	NULL	
job_role	varchar(30)	YES		NULL	

INSERTION OF VALUES:

INSERT INTO dept VALUES (10,'ACCOUNTING','MUMBAI');
INSERT INTO dept VALUES (20,'RESEARCH','BENGALUR');
INSERT INTO dept VALUES (30,'SALES','DELHI');
INSERT INTO dept VALUES (40,'OPERATIONS','CHENNAI');

select * from dept;

deptno	dname	loc
10	ACCOUNTING	MUMBAI
20	RESEARCH	BENGALURU
30	SALES	DELHI
40	OPERATIONS	CHENNAI
NULL	NULL	NULL

INSERT INTO emp VALUES (7369,'Adarsh',7902,'2012-12-17','80000.00','20');
INSERT INTO emp VALUES (7499,'Shruthi',7698,'2013-02-20','16000.00','30');
INSERT INTO emp VALUES (7521,'Anvitha',7698,'2015-02-22','12500.00','30');
INSERT INTO emp VALUES (7566,'Tanvir',7839,'2008-04-02','29750.00','20');
INSERT INTO emp VALUES (7654,'Ramesh',7698,'2014-09-28','12500.00','30');
INSERT INTO emp VALUES (7698,'Kumar',7839,'2015-05-01','28500.00','30');
INSERT INTO emp VALUES (7782,'CLARK',7839,'2017-06-09','24500.00','10');
INSERT INTO emp VALUES (7788,'SCOTT',7566,'2010-12-09','30000.00','20');
INSERT INTO emp VALUES ('7839','KING',NULL,'2009-11-17','50000.00','10');
INSERT INTO emp VALUES ('7844','TURNER',7698,'2010-09-08','15000.00','30');
INSERT INTO emp VALUES ('7876','ADAMS',7788,'2013-01-12','11000.00','20');
INSERT INTO emp VALUES ('7900','JAMES',7698,'2017-12-03','9500.00','30');
INSERT INTO emp VALUES ('7902','FORD','7566','2010-12-03','30000.00','20');

select * from emp;

empno	ename	mgr_no	hiredate	sal	deptno
7369	Adarsh	7902	2012-12-17	80000.00	20
7499	Shruthi	7698	2013-02-20	16000.00	30
7521	Anvitha	7698	2015-02-22	12500.00	30
7566	Tanvir	7839	2008-04-02	29750.00	20
7654	Ramesh	7698	2014-09-28	12500.00	30
7698	Kumar	7839	2015-05-01	28500.00	30
7782	CLARK	7839	2017-06-09	24500.00	10
7788	SCOTT	7566	2010-12-09	30000.00	20
7839	KING	NULL	2009-11-17	50000.00	10
7844	TURNER	7698	2010-09-08	15000.00	30
7876	ADAMS	7788	2013-01-12	11000.00	20
7900	JAMES	7698	2017-12-03	9500.00	30
7902	FORD	7566	2010-12-03	30000.00	20
NULL	NULL	NULL	NULL	NULL	NULL

INSERT INTO incentives VALUES(7499,'2019-02-01',5000.00);
INSERT INTO incentives VALUES(7521,'2019-03-01',2500.00);
INSERT INTO incentives VALUES(7566,'2022-02-01',5070.00);
INSERT INTO incentives VALUES(7654,'2020-02-01',2000.00);
INSERT INTO incentives VALUES(7654,'2022-04-01',879.00);
INSERT INTO incentives VALUES(7521,'2019-02-01',8000.00);
INSERT INTO incentives VALUES(7698,'2019-03-01',500.00);
INSERT INTO incentives VALUES(7698,'2020-03-01',9000.00);
INSERT INTO incentives VALUES(7698,'2022-04-01',4500.00);

select * from incentives;

empno	incentive_date	incentive_amount
7499	2019-02-01	5000.00
7521	2019-02-01	8000.00
7521	2019-03-01	2500.00
7566	2022-02-01	5070.00
7654	2020-02-01	2000.00
7654	2022-04-01	879.00
7698	2019-03-01	500.00
7698	2020-03-01	9000.00
7698	2022-04-01	4500.00
NULL	NULL	NULL

INSERT INTO project VALUES(101,'AI Project','BENGALURU');
INSERT INTO project VALUES(102,'IOT','HYDERABAD');
INSERT INTO project VALUES(103,'BLOCKCHAIN','BENGALURU');
INSERT INTO project VALUES(104,'DATA SCIENCE','MYSURU');
INSERT INTO project VALUES(105,'AUTONOMUS SYSTEMS','PUNE');

select * from project;

pno	pname	ploc
101	AI Project	BENGALURU
102	IOT	HYDERABAD
103	BLOCKCHAIN	BENGALURU
104	DATA SCIENCE	MYSURU
105	AUTONOMOUS SYSTEMS	PUNE
NULL	NULL	NULL

```
INSERT INTO assigned_to VALUES(7499,101,'Software Engineer');
INSERT INTO assigned_to VALUES(7521,101,'Software Architect');
INSERT INTO assigned_to VALUES(7566,101,'Project Manager');
INSERT INTO assigned_to VALUES(7654,102,'Sales');
INSERT INTO assigned_to VALUES(7521,102,'Software Engineer');
INSERT INTO assigned_to VALUES(7499,102,'Software Engineer');
INSERT INTO assigned_to VALUES(7654,103,'Cyber Security');
INSERT INTO assigned_to VALUES(7698,104,'Software Engineer');
INSERT INTO assigned_to VALUES(7839,104,'General Manager');
INSERT INTO assigned_to VALUES(7900,105,'Software Engineer');
```

```
select * from assigned_to;
```

empno	pno	job_role
7499	101	Software Engineer
7499	102	Software Engineer
7521	101	Software Architect
7521	102	Software Engineer
7566	101	Project Manager
7654	102	Sales
7654	103	Cyber Security
7698	104	Software Engineer
7839	104	General Manager
7900	105	Software Engineer
NULL	NULL	NULL

QUERY 1: Retrieve the employee numbers of all employees who work on project located in Bengaluru, Hyderabad, or Mysuru

```
select e.empno from emp e,project p,assigned_to a where e.empno=a.empno
                and a.pno=p.pno and p.ploc in('BENGALURU','HYDERABAD','MYSURU');
```

empno
7499
7499
7521
7521
7566
7654
7654
7698
7839

QUERY 2: Get Employee ID's of those employees who didn't receive incentives

```
select empno from emp where empno not in (select empno from incentives);
```

empno
7369
7782
7788
7839
7844
7876
7900
7902
NULL

QUERY 3: Write a SQL query to find the employees name, number, dept, job_role, department location and project location who are working for a project location same as his/her department location.

```
select e.empno , e.ename,d.dname,d.loc,a.job_role,p.ploc  from emp e,dept d,
assigned_to a,project p where e.deptno=d.deptno and  e.empno=a.empno and
a.pno=p.pno and d.loc=p.ploc;
```

empno	ename	dname	loc	job_role	ploc
7566	Tanvir	RESEARCH	BENGALURU	Project Manager	BENGALURU

QUERY 4: Increase income of all employees by 5% in a table

```
update emp set sal = 1.05*sal;
```

select * from emp;

empno	ename	mgr_no	hiredate	sal	deptno
7369	Adarsh	7902	2012-12-17	84000.00	20
7499	Shruthi	7698	2013-02-20	16800.00	30
7521	Anvitha	7698	2015-02-22	13125.00	30
7566	Tanvir	7839	2008-04-02	31237.50	20
7654	Ramesh	7698	2014-09-28	13125.00	30
7698	Kumar	7839	2015-05-01	29925.00	30
7782	CLARK	7839	2017-06-09	25725.00	10
7788	SCOTT	7566	2010-12-09	31500.00	20
7839	KING	NULL	2009-11-17	52500.00	10
7844	TURNER	7698	2010-09-08	15750.00	30
7876	ADAMS	7788	2013-01-12	11550.00	20
7900	JAMES	7698	2017-12-03	9975.00	30
7902	FORD	7566	2010-12-03	31500.00	20
NULL	NULL	NULL	NULL	NULL	NULL

<u>QUERY 5:</u> Find the name of employees staring with ‘A’

select ename from emp where ename like 'A%';

ename
Adarsh
Anvitha
ADAMS

WEEK 6 – ADDITIONAL QUERIES – (Emp. Database)

QUERY 1: List the name of managers with most employees

```
> SELECT m.ename, count(*) FROM emp e,emp m WHERE e.mgr_no = m.empno
GROUP BY m.ename HAVING count(*) =(SELECT MAX(mycount)
                                  from (SELECT COUNT(*) mycount
                                  FROM emp
                                  GROUP BY mgr_no) a);
```

Result Grid | Filter R

	ename	count(*)
▶	Kumar	5

QUERY 2: Display those managers name whose salary is more than the average salary f his employees

```
> SELECT * FROM emp m WHERE m.empno IN (SELECT mgr_no FROM emp) AND m.sal >
(SELECT avg(e.sal) FROM emp e WHERE e.mgr_no = m.empno );
```

Result Grid | Filter Rows: | Edit:

	empno	ename	mgr_no	hiredate	sal	deptno
▶	7698	Kumar	7839	2015-05-01	28500.00	30
	7839	KING	NULL	2009-11-17	50000.00	10
	7788	SCOTT	7566	2010-12-09	30000.00	20
*	NULL	NULL	NULL	NULL	NULL	NULL

QUERY 3: Write SQL Query to find the top level manager of each department

```
> select distinct m.mgr_no from emp e,emp m
  where e.mgr_no=m.mgr_no and e.deptno=m.deptno and e.empno in (
  select distinct m.mgr_no from emp e,emp m
  where e.mgr_no=m.mgr_no and e.deptno=m.deptno);
```

Result Grid | Filter Rows: | Edit:

	empno	ename	mgr_no	hiredate	sal	deptno
▶	7566	Tanvir	7839	2008-04-02	29750.00	20
	7839	KING	NULL	2009-11-17	50000.00	10
*	NULL	NULL	NULL	NULL	NULL	NULL

QUERY 4: Display those employee who are working in same department where his/her manager is working

```
> SELECT * FROM emp E WHERE E.DEPTNO = (SELECT E1.DEPTNO FROM emp E1 WHERE
E1.EMPNO=E.MGR_NO);
```

Result Grid | Filter Rows: | Edit:

empno	ename	mgr_no	hiredate	sal	deptno
7369	Adarsh	7902	2012-12-17	80000.00	20
7499	Shruthi	7698	2013-02-20	16000.00	30
7521	Anvitha	7698	2015-02-22	12500.00	30
7654	Ramesh	7698	2014-09-28	12500.00	30
7782	CLARK	7839	2017-06-09	24500.00	10
7788	SCOTT	7566	2010-12-09	30000.00	20
7844	TURNER	7698	2010-09-08	15000.00	30
7876	ADAMS	7788	2013-01-12	11000.00	20
7900	JAMES	7698	2017-12-03	9500.00	30
7902	FORD	7566	2010-12-03	30000.00	20
NULL	NULL	NULL	NULL	NULL	NULL

QUERY 5: SQL Query to find employee details who got second highest incentive in February 2019

> select * from emp e,incentives I where
e.empno=i.empno and 2 = (select count(*) from incentives j where
i.incentive_amount <= j.incentive_amount);

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empno	ename	mgr_no	hiredate	sal	deptno	empno	incentive_date	incentive_amount
7521	Anvitha	7698	2015-02-22	12500.00	30	7521	2019-02-01	8000.00

QUERY 6: Write a SQL query to find those employees whose net pay are higher than or equal to the salary of any other employee in the company.

> SELECT distinct e.ename FROM emp e,incentives I
WHERE (SELECT max(sal+incentive_amount)
FROM emp,incentives) <= ANY
(SELECT sal FROM emp e1 where e.deptno=e1.deptno);

ename
Adarsh
Shruthi
Anvitha
Tanvir
Ramesh
Kumar
CLARK
SCOTT
KING
TURNER
ADAMS
JAMES
FORD

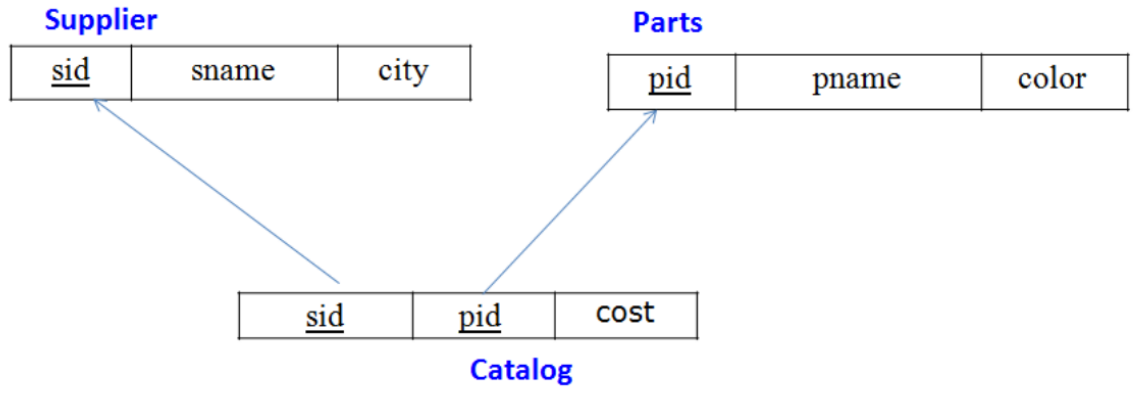
WEEK 7 - SUPPLIER DATABASE

QUESTION:

Consider the following schema:
SUPPLIERS(sid: integer, sname: string, address: string)
PARTS(pid: integer, pname: string, color: string)
CATALOG(sid: integer, pid: integer, cost: real)
The Catalog relation lists the prices charged for parts by Suppliers.

Write the following queries in SQL:
i) Find the pnames of parts for which there is some supplier.
ii) Find the snames of suppliers who supply every part.
iii) Find the snames of suppliers who supply every red part.
iv) Find the pnames of parts supplied by Acme Widget Suppliers and by no one else.
v) Find the sids of suppliers who charge more for some part than the average cost of that
part (averaged over all the suppliers who supply that part).
vi) For each part, find the sname of the supplier who charges the most for that part.

Schema Diagram



CREATION OF DATABASE:

>create database supplier;
> use supplier;

CREATION OF TABLES:

```
CREATE TABLE suppliers (
   sid integer(5) primary key,
   sname varchar(20),
   city varchar(20));

CREATE TABLE parts (
   pid integer(5) primary key,
   pname varchar(20),
   color varchar(10));

CREATE TABLE catalog (
   sid integer(5),
   pid integer(5),
   cost float(6),
   foreign key (sid) references suppliers(sid),
   foreign key (pid) references parts(pid),
   primary key (sid, pid));
```

STRUCTURE OF TABLES:

desc suppliers;

Result Grid | Filter Rows: | Export: | Wrap C

Field	Type	Null	Key	Default	Extra
sid	int	NO	PRI	NULL	
sname	varchar(20)	YES		NULL	
city	varchar(20)	YES		NULL	

desc parts;

Result Grid | Filter Rows: | Export: | W

Field	Type	Null	Key	Default	Extra
pid	int	NO	PRI	NULL	
pname	varchar(20)	YES		NULL	
color	varchar(10)	YES		NULL	

desc catalog;

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
sid	int	NO	PRI	NULL	
pid	int	NO	PRI	NULL	
cost	float	YES		NULL	

INSERTION OF VALUES:

```
insert into suppliers values (10001, 'acme widget', 'bangalore');
insert into suppliers values (10002, 'johns', 'kolkata');
insert into suppliers values (10003, 'vimal', 'mumbai');
insert into suppliers values (10004, 'reliance', 'delhi');
insert into suppliers values (10005, 'mahindra', 'mumbai');
```

```
select * from suppliers;
```

Result Grid | Filter Rows:

sid	sname	city
10001	acme widget	bangalore
10002	johns	kolkata
10003	vimal	mumbai
10004	reliance	delhi
10005	mahindra	mumbai
NULL	NULL	NULL

```
insert into parts values (20001, 'book', 'red');
insert into parts values (20002, 'pen', 'red');
insert into parts values (20003, 'pencil', 'green');
insert into parts values (20004, 'mobile', 'green');
insert into parts values (20005, 'charger', 'black');
```

```
select * from parts;
```

Result Grid | Filter Rows:

pid	pname	color
20001	book	red
20002	pen	red
20003	pencil	green
20004	mobile	green
20005	charger	black
NULL	NULL	NULL

```
insert into catalog values (10001, 20001, 10);
insert into catalog values (10001, 20002, 10);
insert into catalog values (10001, 20003, 30);
insert into catalog values (10001, 20004, 10);
insert into catalog values (10001, 20005, 10);
insert into catalog values (10002, 20001, 10);
insert into catalog values (10002, 20002, 20);
insert into catalog values (10003, 20003, 30);
insert into catalog values (10004, 20003, 40);
```

select * from catalog;

Result Grid | Filter Rows:

sid	pid	cost
10001	20001	10
10001	20002	10
10001	20003	30
10001	20004	10
10001	20005	10
10002	20001	10
10002	20002	20
10003	20003	30
10004	20003	40
NULL	NULL	NULL

QUERY 1: Find the pnames of parts for which there is some supplier.

> select pname from parts where pid in (select pid from catalog);

Result Grid

pname
book
pen
pencil
mobile
charger

QUERY 2: Find the snames of suppliers who supply every part.

> select sname from supplier where sid in (select sid from cataloggroup by sid having count(distinct pid) = (select count(distinct pid) from parts));

Result Grid

sname
acme widget

QUERY 3: Find the supplier IDs of suppliers who charge more for some part than the average cost of that part (averaged over all the suppliers who supply that part).

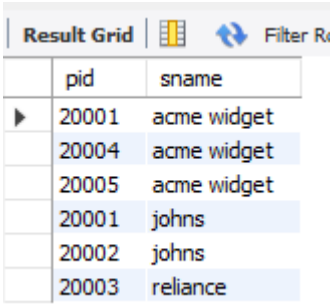
> SELECT sid FROM catalog a WHERE a.cost > (SELECT AVG(b.cost) FROM catalog b WHERE a.pid = b.pid GROUP BY b.pid);

Result Grid

sid
10002
10004

QUERY 4: For each part, find the supplier name who charges the most for that part.

> SELECT pid, sname FROM catalog a, suppliers WHERE a.cost = (SELECT MAX(b.cost) FROM catalog b WHERE a.pid = b.pid GROUP BY b.pid) AND suppliers.sid = a.sid;

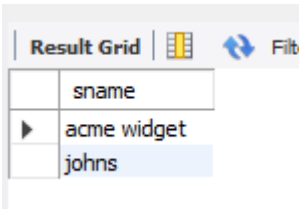


Result Grid | Filter R

	pid	sname
▶	20001	acme widget
	20004	acme widget
	20005	acme widget
	20001	johns
	20002	johns
	20003	reliance

QUERY 5: Find the snames of suppliers who supply every red part.

> SELECT DISTINCT sname FROM suppliers, parts, catalog WHERE suppliers.sid = catalog.sid AND parts.pid = catalog.pid AND parts.color = 'Red';

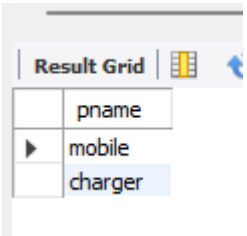


Result Grid | Filt

	sname
▶	acme widget
	johns

QUERY 6: Find the part names of parts supplied by Acme Widget Suppliers and by no one else.

> SELECT pname FROM parts WHERE pid NOT IN (SELECT pid FROM catalog WHERE sid IN (SELECT sid FROM suppliers WHERE sname != 'Acme Widget'));



Result Grid

	pname
▶	mobile
	charger

WEEK 8 – NO SQL LAB 1

Question

Perform the following DB operations using MongoDB.
1. Create a database “Student” with the following attributes Rollno, Age, ContactNo, Email Id.
2. Insert appropriate values
3. Write query to update Email-Id of a student with rollno 10.
4. Replace the student name from “ABC” to “FEM” of rollno 11.
5. Export the created table into local file system
6. Drop the table
7. Import a given csv dataset from local file system into mongodb collection.

1. CREATE DATABASE:

db.createCollection("Student");

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.createCollection("Student");
{ ok: 1 }
```

2. CREATE TABLE & INSERTING VALUES TO THE TABLE:

```
db.Student.insert({RollNo:1,Age:21,Cont:9876,email:"antara.de9@gmail.com"});
db.Student.insert({RollNo:2,Age:22,Cont:9976,email:"anushka.de9@gmail.com"});
db.Student.insert({RollNo:3,Age:21,Cont:5576,email:"anubhav.de9@gmail.com"});
db.Student.insert({RollNo:4,Age:20,Cont:4476,email:"pani.de9@gmail.com"});
db.Student.insert({RollNo:10,Age:23,Cont:2276,email:"rekha.de9@gmail.com"});
```

```
db.Student.find();
```

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Student.find();
[
  {
    _id: ObjectId('676142507a54f136bd9f990a'),
    RollNo: 1,
    Age: 21,
    Cont: 9876,
    email: 'antara.de9@gmail.com'
  },
  {
    _id: ObjectId('676144787a54f136bd9f990b'),
    RollNo: 2,
    Age: 22,
    Cont: 9976,
    email: 'anushka.de9@gmail.com'
  },
  {
    _id: ObjectId('676144867a54f136bd9f990c'),
    RollNo: 3,
    Age: 21,
    Cont: 5576,
    email: 'anubhav.de9@gmail.com'
  },
  {
    _id: ObjectId('676144927a54f136bd9f990d'),
    RollNo: 4,
    Age: 20,
    Cont: 4476,
    email: 'pani.de9@gmail.com'
  },
  {
    _id: ObjectId('6761449d7a54f136bd9f990e'),
    RollNo: 10,
    Age: 23,
    Cont: 2276,
    email: 'rekha.de9@gmail.com'
  }
]
```

3. Write query to update Email-Id of a student with rollno 10.

db.Student.update({RollNo:10},{$set:{ email:"Abhinav@gmail.com"}})

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Student.update({RollNo:10},{$set:{
... email:"Abhinav@gmail.com"}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. Replace the student's name from "ABC" to "FEM" of rollno 11.

db.Student.insert({RollNo:11,Age:22,Name:"ABC",Cont:2276,email:"rea.de9@gmail.com"});
db.Student.update({RollNo:11,Name:"ABC"},{$set:{Name:"FEM"}})

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Student.insert({RollNo:11,Age:22,Name:"ABC",Cont:2276,email:"rea.de9@gmail.com"});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('676146667a54f136bd9f990f') }
}
```

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Student.update({RollNo:11,Name:"ABC"},{$set:{Name:"FEM"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. Export the created table into local file system

mongoexport mongodb+srv://antararc:****@cluster0.mfnfeys.mongodb.net/myDB --collection=Student –out C:\Users\BMSCECSE\Downloads\output.json

	A	B	C	D	E	F
1	_id	RollNo	Age	Cont	email	Name
2	676142507a54f136bd9f990a	1	21	9876	antara.de9@gmail.com	
3	676144787a54f136bd9f990b	2	22	9976	anushka.de9@gmail.com	
4	676144867a54f136bd9f990c	3	21	5576	anubhav.de9@gmail.com	
5	676144927a54f136bd9f990d	4	20	4476	pani.de9@gmail.com	
6	6761449d7a54f136bd9f990e	10	23	2276	Abhinav@gmail.com	
7	676146667a54f136bd9f990f	11	22	2276	rea.de9@gmail.com	FEM
8						

6. Drop the table

db.Student.drop();

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Student.drop();
true
Atlas atlas-2x41vp-shard-0 [primary] test>
```

7. Import a given csv dataset from local file system into mongodb collection.

mongoimport mongodb+srv://antararc:Test1234@cluster0.mfnfeys.mongodb.net/myDB --collection=New_Student --type json --file C:\Users\BMSCECSE\Downloads\output.json

db.Students.find();

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Student.find();
[
  {
    _id: ObjectId('676142507a54f136bd9f990a'),
    RollNo: 1,
    Age: 21,
    Cont: 9876,
    email: 'antara.de9@gmail.com'
  },
  {
    _id: ObjectId('676144787a54f136bd9f990b'),
    RollNo: 2,
    Age: 22,
    Cont: 9976,
    email: 'anushka.de9@gmail.com'
  },
  {
    _id: ObjectId('676144867a54f136bd9f990c'),
    RollNo: 3,
    Age: 21,
    Cont: 5576,
    email: 'anubhav.de9@gmail.com'
  },
  {
    _id: ObjectId('676144927a54f136bd9f990d'),
    RollNo: 4,
    Age: 20,
    Cont: 4476,
    email: 'pani.de9@gmail.com'
  },
  {
    _id: ObjectId('6761449d7a54f136bd9f990e'),
    RollNo: 10,
    Age: 23,
    Cont: 2276,
    email: 'Abhinav@gmail.com'
  },
  {
    _id: ObjectId('676146667a54f136bd9f990f'),
    RollNo: 11,
    Age: 22,
    Cont: 2276,
    email: 'rea.de9@gmail.com',
    Name: 'FEM'
  }
]
```

WEEK 9 – NO SQL LAB 2

Perform the following DB operations using MongoDB.

1. Create a collection by name Customers with the following attributes. Cust_id, Acc_Bal, Acc_Type
2. Insert at least 5 values into the table
3. Write a query to display those records whose total account balance is greater than 1200 of account type ‘Z’ for each customer_id.
4. Determine Minimum and Maximum account balance for each customer_id.
5. Export the created collection into local file system
6. Drop the table
7. Import a given csv dataset from local file system into mongodb collection.

1. Create a collection by name Customers with the following attributes. Cust_id, Acc_Bal, Acc_Type

```
db.createCollection("Customer");
```

2. Insert at least 5 values into the table

```
db.Customer.insertMany([{custid: 1, acc_bal:10000, acc_type: "Saving"}, {custid: 1, acc_bal:20000, acc_type: "Checking"}, {custid: 3, acc_bal:50000, acc_type: "Checking"}, {custid: 4, acc_bal:10000, acc_type: "Saving"}, {custid: 5, acc_bal:2000, acc_type: "Checking"}]);
```

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Customer.insertMany([
...   { custid: 1, acc_bal: 10000, acc_type: "Saving" },
...   { custid: 1, acc_bal: 20000, acc_type: "Checking" },
...   { custid: 3, acc_bal: 50000, acc_type: "Checking" },
...   { custid: 4, acc_bal: 10000, acc_type: "Saving" },
...   { custid: 5, acc_bal: 2000, acc_type: "Checking" }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67614b337a54f136bd9f9910'),
    '1': ObjectId('67614b337a54f136bd9f9911'),
    '2': ObjectId('67614b337a54f136bd9f9912'),
    '3': ObjectId('67614b337a54f136bd9f9913'),
    '4': ObjectId('67614b337a54f136bd9f9914')
  }
}
```

3. Finding all checking accounts with balance greater than 12000 and acc_type “checking”

```
db.Customer.find({acc_bal: {$gt: 12000}, acc_type:"Checking"});
```

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Customer.find({acc_bal: {$gt: 12000}, acc_type:"Checking"});
[
  {
    _id: ObjectId('67614b337a54f136bd9f9911'),
    custid: 1,
    acc_bal: 20000,
    acc_type: 'Checking'
  },
  {
    _id: ObjectId('67614b337a54f136bd9f9912'),
    custid: 3,
    acc_bal: 50000,
    acc_type: 'Checking'
  }
]
```

4. Finding the maximum and minimum balance of each customer

db.Customer.aggregate([{$group:{_id:"$custid", minBal:{$min:"$acc_bal"}, maxBal: {$max:"$acc_bal"}}}]);

```
[
  { _id: 1, minBal: 10000, maxBal: 20000 },
  { _id: 5, minBal: 2000, maxBal: 2000 },
  { _id: 3, minBal: 50000, maxBal: 50000 },
  { _id: 4, minBal: 10000, maxBal: 10000 }
]
```

5. Export the created collection into local file system

mongoexport mongodb+srv://204:<password>@cluster0.xbmgopf.mongodb.net/test

--collection=Customer -- out type json --file C:\Users\BMSCECSE\Downloads\output.json

	A	B	C	D
1	_id	custid	acc_bal	acc_type
2	67614b337a54f136bd9f9910	1	10000	Saving
3	67614b337a54f136bd9f9911	1	20000	Checking
4	67614b337a54f136bd9f9912	3	50000	Checking
5	67614b337a54f136bd9f9913	4	10000	Saving
6	67614b337a54f136bd9f9914	5	2000	Checking
7				

6. Drop the table:

db.Customer.drop();

```
Atlas atlas-2x41vp-shard-0 [primary] test> db.Customer.drop();
true
Atlas atlas-2x41vp-shard-0 [primary] test>
```

7. Import a given csv dataset from local file system into mongodb collection.

mongoimport mongodb+srv://antararc:Test1234@cluster0.mfnfeys.mongodb.net/myDB --collection=New_Customer --type json --file C:\Users\BMSCECSE\Downloads\output.json

db.Customer.find();

```
test> db.Customer.find();
[
  {
    _id: ObjectId('65e418fc5b3b1935aac1fe4b'),
    custid: 1,
    acc_bal: 10000,
    acc_type: 'Saving'
  },
  {
    _id: ObjectId('65e418fc5b3b1935aac1fe4c'),
    custid: 1,
    acc_bal: 20000,
    acc_type: 'Checking'
  },
  {
    _id: ObjectId('65e418fc5b3b1935aac1fe4d'),
    custid: 3,
    acc_bal: 50000,
    acc_type: 'Checking'
  },
  {
    _id: ObjectId('65e418fc5b3b1935aac1fe4e'),
    custid: 4,
    acc_bal: 10000,
    acc_type: 'Saving'
  },
  {
    _id: ObjectId('65e418fc5b3b1935aac1fe4f'),
    custid: 5,
    acc_bal: 2000,
    acc_type: 'Checking'
  }
]
```

WEEK 10 – NO SQL LAB 3

QUESTION:

1. Write a MongoDB query to display all the documents in the

collection restaurants.

2. Write a MongoDB query to arrange the name of the restaurants in

descending along with all the columns.

3. Write a MongoDB query to find the restaurant Id, name, town and

cuisine for those restaurants which achieved a score which is not

more than 10.

4. Write a MongoDB query to find the average score for each

restaurant.

5. Write a MongoDB query to find the name and address of the

restaurants that have a zipcode that starts with '10'.

CREATE Table:

```
db.createCollection("restaurants");
```

Inserting Values:

```
db.restaurants.insertMany([
{ name: "Meghna Foods", town: "Jayanagar", cuisine: "Indian", score: 8, address: { zipcode:
"10001",
street: "Jayanagar“
} },
{ name: "Empire", town: "MG Road", cuisine: "Indian", score: 7, address: { zipcode: "10100",
street: "MG
Road" } },
{ name: "Chinese WOK", town: "Indiranagar", cuisine: "Chinese", score: 12, address: {
zipcode: "20000",
street: "Indiranagar" } },
{ name: "Kyotos", town: "Majestic", cuisine: "Japanese", score: 9, address: { zipcode:
"10300", street:
"Majestic" } },
{ name: "WOW Momos", town: "Malleshwaram", cuisine: "Indian", score: 5, address: {
zipcode: "10400",
44
street: "Malleshwaram" }
} ])
```

1. Write a MongoDB query to display all the documents in the collection restaurants.

db.restaurants.find();

```
Atlas atlas-13yfay-shard-0 [primary] test> db.restaurants.find({})
[
  {
    _id: ObjectId("67500261f345f747889620b9"),
    name: 'Meghna Foods',
    town: 'Jayanagar',
    cuisine: 'Indian',
    score: 8,
    address: { zipcode: '10001', street: 'jayanagar' }
  },
  {
    _id: ObjectId("67500292f345f747889620ba"),
    name: 'Empire',
    town: 'M G Road',
    cuisine: 'Indian',
    score: 7,
    address: { zipcode: '10100', street: 'M G Road' }
  },
  {
    _id: ObjectId("675002dbf345f747889620bb"),
    name: 'Chinese Wok',
    town: 'Indiranagar',
    cuisine: 'Chinese',
    score: 12,
    address: { zipcode: '20000', street: 'Indiranagar' }
  },
  {
    _id: ObjectId("67500316f345f747889620bc"),
    name: 'Kyotos',
    town: 'Majestic',
    cuisine: 'japanese',
    score: 9,
    address: { zipcode: '10300', street: 'Majestic' }
  },
  {
    _id: ObjectId("67500342f345f747889620bd"),
    name: 'WOW Momo',
    town: 'Malleshwaram',
    cuisine: 'Indian',
    score: 5,
    address: { zipcode: '10400', street: 'Malleshwaram' }
  }
]
```

2. Write a MongoDB query to arrange the name of the restaurants in descending along with all the columns.

db.restaurants.find({}).sort({ name: -1 })

```
Atlas atlas-13yfay-shard-0 [primary] test> db.restaurants.find({}).sort({name:-1})
[
  {
    _id: ObjectId("67500342f345f747889620bd"),
    name: 'WOW Momo',
    town: 'Malleshwaram',
    cuisine: 'Indian',
    score: 5,
    address: { zipcode: '10400', street: 'Malleshwaram' }
  },
  {
    _id: ObjectId("67500261f345f747889620b9"),
    name: 'Meghna Foods',
    town: 'Jayanagar',
    cuisine: 'Indian',
    score: 8,
    address: { zipcode: '10001', street: 'jayanagar' }
  },
  {
    _id: ObjectId("67500316f345f747889620bc"),
    name: 'Kyotos',
    town: 'Majestic',
    cuisine: 'japanese',
    score: 9,
    address: { zipcode: '10300', street: 'Majestic' }
  },
  {
    _id: ObjectId("67500292f345f747889620ba"),
    name: 'Empire',
    town: 'M G Road',
    cuisine: 'Indian',
    score: 7,
    address: { zipcode: '10100', street: 'M G Road' }
  },
  {
    _id: ObjectId("675002dbf345f747889620bb"),
    name: 'Chinese Wok',
    town: 'Indiranagar',
    cuisine: 'Chinese',
    score: 12,
    address: { zipcode: '20000', street: 'Indiranagar' }
  }
]
```

3. Write a MongoDB query to find the restaurant Id, name, town and cuisine for those restaurants which achieved a score which is not more than 10.

db.restaurants.find({ "score": { $lte: 10 } }, { _id: 1, name: 1, town: 1, cuisine: 1 })

```
[
  {
    _id: ObjectId("67500261f345f747889620b9"),
    name: 'Meghna Foods',
    town: 'Jayanagar',
    cuisine: 'Indian'
  },
  {
    _id: ObjectId("67500292f345f747889620ba"),
    name: 'Empire',
    town: 'M G Road',
    cuisine: 'Indian'
  },
  {
    _id: ObjectId("67500316f345f747889620bc"),
    name: 'Kyotos',
    town: 'Majestic',
    cuisine: 'japanese'
  },
  {
    _id: ObjectId("67500342f345f747889620bd"),
    name: 'WOW Momo',
    town: 'Malleshwaram',
    cuisine: 'Indian'
  }
]
```

4. Write a MongoDB query to find the average score for each restaurant.

db.restaurants.aggregate([{ $group: { _id: "$name", average_score: { $avg: "$score" } } }])

```
Atlas atlas-13yfay-shard-0 [primary] test> db.restaurants.aggregate([ { $group: { _id: "$name", average_score: { $avg: "$score" } } }
... ])
[
  { _id: 'WOW Momo', average_score: 5 },
  { _id: 'Meghna Foods', average_score: 8 },
  { _id: 'Kyotos', average_score: 9 },
  { _id: 'Chinese Wok', average_score: 12 },
  { _id: 'Empire', average_score: 7 }
]
```

5. Write a MongoDB query to find the name and address of the restaurants that have a zipcode that starts with '10'.

db.restaurants.find({ "address.zipcode": /^10/ }, { name: 1, "address.street": 1, _id: 0 })

```
Atlas atlas-13yfay-shard-0 [primary] test> db.restaurants.find({ "address.zipcode": /^10/ }, { name: 1, "address.street": 1, _id: 0 })
[
  { name: 'Meghna Foods', address: { street: 'jayanagar' } },
  { name: 'Empire', address: { street: 'M G Road' } },
  { name: 'Kyotos', address: { street: 'Majestic' } },
  { name: 'WOW Momo', address: { street: 'Malleshwaram' } }
]
```